

RAI: Safe Pipelined Epoch Transitions for Account-Local Payments

Supplementary Material

Anonymous Author(s)

Scope and reading guide

This supplement accompanies the main manuscript, which specifies RAI over two abstract services: a convergence service that reconstructs frozen reports and a consensus service that decides one eligible checkpoint per epoch. The main manuscript is self-contained for the transition rules, the eligibility predicate, and the safety and termination arguments relative to those services. Appendix A describes the consensus instantiation used by the prototype of the main manuscript’s Section 9, a Kudzu-style election, and argues that it provides properties A1–A4 of the main manuscript’s Section 3.5; it is one instantiation, not part of the protocol’s definition. Appendix B makes explicit the temporal premise needed by predecessor-backed fresh-work voting; its local exclusion lemma is not presented as a complete composition proof for that extension. Appendix C records deterministic-construction and overlap cases that an independent formal audit should cover. The convergence instantiation, rateless set reconciliation, is described in the main manuscript only; any mechanism satisfying C1 and C2 may replace it.

Throughout, $N = 3f + 2p + 1$, $q = N - f - p = 2f + p + 1$, and $r = f + p + 1$. At most f identities of a committee are Byzantine under the globally static corruption model. Account votes and election votes have separate domains. Election notarization can use second-look support, unlike the single-support account path. All election signatures bind the handoff context, deciding committee, election slot, vote type, and proposal hash or timeout value.

A The Prototype’s Consensus Instantiation: A Single-Kudzu Checkpoint Election

A.1 Proposal history and candidate validity

Election slots $w = 1, 2, \dots$ are distinct from account slots. Leaders rotate through the closing committee C_{e-2} . Election genesis is slot zero. A non-timeout proposal binds the handoff context, election slot, parent hash, complete payload (Q, μ_e, d_e) , and leader signature. Its parent occupies an earlier slot and has a valid election NC; every skipped intervening slot has a TC.

Only a child of genesis can introduce a fresh payload. Its leader chooses $N - f$ distinct usable report headers, pins

the evidence manifest by μ_e , and obtains d_e by deterministic checkpoint construction. A descendant of a certified non-genesis parent copies all payload fields, including the evidence commitment. It cannot change the report selection or substitute a new proof inventory. Different fresh candidates may be valid, but a decided history preserves one payload.

Before a correct validator supports a non-timeout proposal, it reconstructs the reports, retrieves the manifest’s data, verifies all semantic claims, runs BuildState, checks the state digest, and validates the proposal ancestry and skipped slots. It retains and serves the authorizing inputs. Validity is fixed by these committed inputs, not by a validator’s unreported account knowledge. An independently acquired account certificate may change a live overlay but not an already committed candidate’s meaning.

The election adapts the Kudzu voting construction [1]. It does not import Kudzu’s erasure-coded payload distribution or claim the corresponding balanced communication bound. Here, full checkpoint input reconstruction is a required preparation step with its own availability and timing costs.

A.2 First votes, second looks, and termination of support

A correct validator first-votes once per election slot. It chooses the first fully valid available leader proposal, or the distinguished timeout value if none is ready by local timeout Δ_E . A first vote also gives notarization support to its value. A normal NC and FC use threshold q ; fast finality uses $N - p$ first votes; a TC uses q distinct timeout-support votes.

Unlike the account path, election voting permits second looks. After first-voting, a still-active validator may support a valid non-timeout proposal with at least r recorded first supporters. It can support timeout when

$$\text{allFirst} - \max_X \text{firstFor}(X) \geq r,$$

where X ranges over non-timeout proposals. Each identity has one recorded first choice in this local tally; later equivocations do not replace it. If no non-timeout first vote is recorded, the maximum is zero. Adding one previously unrecorded sender cannot reduce this difference: total increases by one and the maximum by at most one.

A complete non-timeout NC lets the validator advance to the next election slot and fixes its candidate parent for that advance. It releases a final vote only if it has supported no

other value in the slot, including timeout. It then irreversibly ends slot support, whether or not it was eligible to final-vote. A TC likewise ends slot support. If first-voting and this completion step are simultaneously enabled, the first-vote step occurs first. First-vote, support, and terminal records are durable; observing a later message does not reset them.

This explicit stop-on-complete-NC transition is important. Progress to another election slot cannot depend on having assembled a final certificate. A slot with split support may make progress through notarization or timeout even when no validator can yet finalize its preferred value. Validators continue to relay data and signed votes after their own support ends. A fast tally does not suppress normal final voting when that step is still enabled.

A.3 Decisions, skips, and parent selection

An FC or FF for a non-timeout election proposal decides its payload. A TC permits a slot to be skipped; timeout never installs a checkpoint. A correct leader selects the highest available complete NC-backed parent for which all intervening slots have TCs, with a canonical hash tie-break when needed. It copies that parent's payload, or forms a fresh valid candidate when the parent is genesis.

For a fresh report selection, a correct leader retains its first fully usable report from each identity; an equivocating reporter cannot repeatedly replace that identity's chosen header. Another leader may validly select a different usable report for the same Byzantine identity. Agreement, rather than a global rule about arrival order, chooses the candidate. Each candidate's own headers and manifest remain immutable and fully verifiable.

The deciding committee is the old committee, C_{e-2} . Committee C_{e-1} independently verifies the resulting decision proof and reconstructs the decided input before installing it. It does not run another checkpoint election. Any correct observer with the required data can perform the same checks.

A.4 Agreement safety

Lemma A.1 (Election finality exclusion). *An FC or FF for a proposal excludes an election NC for any other value in the same slot, including timeout.*

Proof. A correct final voter has supported only its selected proposal and supports nothing after terminating the slot. An FC quorum and a conflicting NC quorum would intersect in at least $2q - N = f + 1$ identities, including a correct final voter, which is impossible.

For fast finality, let $f' \leq f$ be the actual Byzantine count. An FF contains at least $N - p - f'$ correct first voters for its proposal. At most p correct first voters remain for all rival values. A rival has at most $p + f' \leq p + f < r$ first supporters and so cannot trigger correct second-look support. Nor can it reach q using first support alone. For timeout's split test, the largest non-timeout count is at least the fast proposal's

count, and all recorded first votes outside that proposal number at most $p + f' < r$. Hence timeout cannot obtain correct second-look support either. The proposal's correct first voters have already validated its full data, so an invalid-data branch is not an additional reason to abandon it. $\square$

Theorem A.2 (Checkpoint agreement). *All valid election decision proofs for one handoff context select the same payload and checkpoint.*

Proof. A final decision in slot w excludes both a conflicting NC and a TC there by Lemma A.1. Any later certified proposal must follow an earlier certified parent and justify every skipped slot. Tracing its parents cannot jump across w , because that would require a TC at w . It must therefore reach the unique compatible NC at w . Every descendant copies the same complete payload. Same-slot conflicting decisions are excluded by the lemma, and the argument applied to the earlier of any two decision slots gives agreement. Fixed-input deterministic construction then yields the same checkpoint state. $\square$

External validity follows because a final quorum contains correct authorizers, each of whom validated the selected reports and fixed manifest. Context binding gives integrity. Retention by correct authorizers provides the data that successors must eventually retrieve. These statements require semantic input verification, not merely signature-count verification of the election proof.

A.5 Prepared-window progress

Lemma A.3 (A prepared slot advances). *During a sufficiently long prepared synchronous interval with $N - f$ responsive correct old validators, a current election slot obtains a complete non-timeout NC or a TC.*

Proof. A correct validator that has already ended support relays the complete NC or TC that let it advance. Otherwise responsive validators eventually first-vote, choosing timeout when no valid proposal is ready. If a non-timeout value has at least r correct first supporters, their validated data are available and all still-active responsive validators can support it, yielding an NC. If no non-timeout value has r correct first supporters, the correct first votes satisfy

$$\text{allFirst} - \max_x \text{firstFor}(X) \geq (N - f) - (r - 1) = r.$$

Adding Byzantine first votes under one recorded choice per identity cannot reduce this difference. Timeout support therefore yields a TC. Data delivery, validation, and the required response stages are included in the prepared interval. $\square$

Theorem A.4 (Conditional election termination). *Suppose valid report and manifest data remain available, processing is fair, and recurring prepared synchronous windows have $N - f$*

responsive correct old members. If the windows cover catch-up, preparation, a full leader cycle, and voting, and

$$\frac{\Delta_E}{1 + \rho} > \kappa_E + \tau_E + \delta,$$

where κ_E bounds slot-entry skew and τ_E bounds proposal preparation and validation, the election eventually decides a checkpoint.

Proof. Correct report reconstruction and finite checkpoint construction supply a valid immutable candidate. The preceding lemma moves voting beyond stalled slots. Round-robin leadership gives a correct proposer within a complete leader cycle. In a fresh prepared correct-leader slot, the timing inequality makes its valid proposal available before any correct first-vote timeout. All responsive correct first votes then support it. Byzantine votes alone cannot trigger a competing second look or the timeout split test. The proposal becomes complete, and at least q correct supporters release final votes when ending the slot, yielding an FC whether or not an FF was already available.

The window must be a common prepared interval, not an arbitrary sum of short synchronous periods. Its budget includes bringing replicas to a useful slot and making the actual candidate data available. After decision, retained-data delivery lets correct successors verify and install the chosen state. No second election is needed. $\square$

B Optional Predecessor-Backed Fresh Work

The main protocol’s late notarization (main manuscript, Section 4.2) already gives fresh successor work closing-epoch exclusion: the outgoing committee notarizes it in its own domain until it installs the predecessor, and same-domain account exclusion applies without any member common to the two committees. The extension below is an alternative for the opposite situation, where the old committee no longer signs but common members carry the exclusion into the successor’s votes.

B.1 Why an unqualified membership intersection is insufficient

The core protocol does not finalize arbitrary fresh successor blocks before predecessor closure. A possible optimization uses common committee members to exclude an old rival at the fresh block’s position. The temporal premise is essential: a member that voted after installing the predecessor checkpoint is no longer subject to the pending-predecessor cross-epoch guard. Membership in both committees alone does not show which guard governed its successor vote.

One concrete way to make the premise part of verifiable evidence is to add a signed first-vote mode. A mode of pending binds the latest verified checkpoint and asserts that the immediate predecessor has not yet been installed.

A mode of installed binds that predecessor’s digest. Correct identities emit pending only while the predecessor is unknown, after durably stopping old signing and checking their persisted old first vote. A signature authenticates the assertion; the behavioral premise follows only for correct signers under the stated fault bound. The network cannot directly observe another process’s knowledge.

The mode does not create a new voting instance. An identity still first-votes once for $(\text{session}, e, a, v)$; it cannot sign one first vote per mode or reset its choice after installation. Votes and their modes must be included in the immutable validation input. This is a proposed message-format refinement, not a property of the unannotated first votes in the original draft.

B.2 A qualified exclusion witness

Definition B.1 (Early predecessor-backed witness). For block B at (a, v) in epoch e , an $\text{EPB}_e(B)$ witness consists of at least q distinct epoch- e first votes for B from identities in K_{e-1} , all signed in the pending mode. Each vote also verifies membership in K_e , its admission context, and the same block and account instance.

Since these q votes are also current-committee first votes, they establish a current NC. Members of $K_e \setminus K_{e-1}$ do not contribute to the predecessor-backed count, although their votes can contribute to an ordinary current certificate. The witness is unavailable if fewer than q common members provide the required pending-mode votes.

Lemma B.2 (Qualified cross-epoch exclusion). *If B has an $\text{EPB}_e(B)$ witness, no different block R at (a, v) has or can obtain an epoch- $(e - 1)$ NC, FC, or FF.*

Proof. Let $A \subseteq K_{e-1}$ be the witness’s q old-member identities, and let $W \subseteq K_{e-1}$ be all old-epoch first voters for R . A correct identity in $A \cap W$ would have first-voted for R in the old epoch and for B in pending mode in the successor. The cross-epoch guard forbids this. A correct member of A also cannot issue an old first vote after its successor vote because it stopped old signing before opening the successor. Thus $|A \cap W| \leq f$, including future old votes, and

$$|W| \leq N - |A| + f \leq N - q + f = 2f + p = q - 1.$$

An old NC requires q first votes. An FF requires at least q , and an FC implies an NC. All are excluded. $\square$

B.3 What is still required for a compositional extension

Lemma B.2 establishes exclusion at one position, not end-to-end transfer safety for an arbitrary fresh descendant. An enabled extension would need to specify a prefix-wide mixture of old/current dual-notarization witnesses and early predecessor-backed witnesses, with a verified anchor and no unsupported intermediate position. It must also show

how selected reports recover latent successor finality when the predecessor-backed witness is itself assembled late.

A complete extension must fix which witness facts are committed, when a recovery-only predecessor lock can be superseded, and how that supersession propagates through inherited lock provenance, BuildState, and the live overlay. In particular, a late-discovered witness cannot silently change an already committed candidate. A proof must cover all valid report selections and all placement orders of predecessor installation, first voting, certificate assembly, and reporting.

The main manuscript does not assume these obligations are discharged by the local intersection argument. Its transfer-safety and termination theorems use the core prefix-wide dual-notarization rule and ordinary post-checkpoint fresh work. The qualified lemma here repairs the temporal statement without claiming that a local repair alone completes the more aggressive protocol.

C Boundary Examples and Audit Cases

C.1 A parent-only overlap counterexample

Take disjoint correct old and successor committees, each satisfying its own fault bound. Let an equivocating account owner sign two children A and B of the same parent P . The old committee notarizes P and certifies A . Independently, the successor re-notarizes P and certifies B , before learning the closing checkpoint. Every within-epoch quorum-intersection rule can hold: the old committee supported only A at the child position and the successor supported only B .

If the sole cross-epoch condition were that P has notarizations in both domains, the two incompatible children could both be treated as final. The missing exclusion is at the children’s position. The core rule closes this case by requiring old-domain notarization for every unresolved position it finalizes, including the certificate target; with late notarization, the old committee supplies that notarization for the child it supports, and within its own domain it cannot notarize both children. Alternatively, a correctly composed predecessor-backed witness could exclude an old rival at that position; merely notarizing P twice cannot.

C.2 A finality witness must not choose the output prefix

Suppose a frozen report tags ancestor B as final. One verifier can validate this using a certificate for B itself; another can validate it using a certificate for descendant D . If the construction automatically inserted every witness’s entire certified prefix, the two verifiers could include different additional blocks even with the same frozen tagged set. The

core specification fixes final-output seeds by named report entries and required finalized-send dependencies. An incidental witness-only descendant is not a new seed. Pinning the witness inventory additionally makes candidate validation reproducible.

C.3 Checkpoint promotion requires notarization

Suppose one candidate retains only block B at the next account position. Raw uniqueness is insufficient: a hidden closing-epoch NC for a rival could support core overlap finality in the successor. Rule 3 therefore promotes B only if B itself has a verified closing-epoch NC; account exclusion then rules out every conflicting closing-epoch NC, and the checkpoint decision can safely make B final. A recovery-only sole survivor remains unresolved.

C.4 Installation cannot replace a live balance with a stale base

Suppose the base checkpoint carries B only as recovery-protected, and a validator subsequently assembles its fast certificate and applies it live. Installing the next checkpoint cannot erase that effect merely because the selected frozen reports have not yet tagged B as final. The union update keeps B in the certified overlay until the base contains it. The same operation identifier appears once in the effective finalized set, even if both a live certificate and a later checkpoint expose it.

C.5 Targets for independent verification

A state-machine model should distinguish certificate existence from local assembly, report freeze from data arrival, and base state from the live overlay. Essential traces include delayed Byzantine votes after checkpoint decision; a final vote just before freeze; mixed T_i/G_i exposure; a withheld or equivocating reporter; old locks with different provenance; attempted promotion of a recovery-only sole survivor; predecessor installation during a provisional attempt; duplicate receives across unresolved ancestors; and a retained target requiring a fresh empty child. These are proposed verification cases, not claims that such a model was executed or that exhaustive testing has established correctness.

References

- [1] Victor Shoup, Jakub Sliwinski, and Yann Vonlanthen. 2025. Kudzu: Fast and Simple High-Throughput BFT. In *39th International Symposium on Distributed Computing (DISC 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 356)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 42:1–42:19. doi:10.4230/LIPIcs.DISC.2025.42